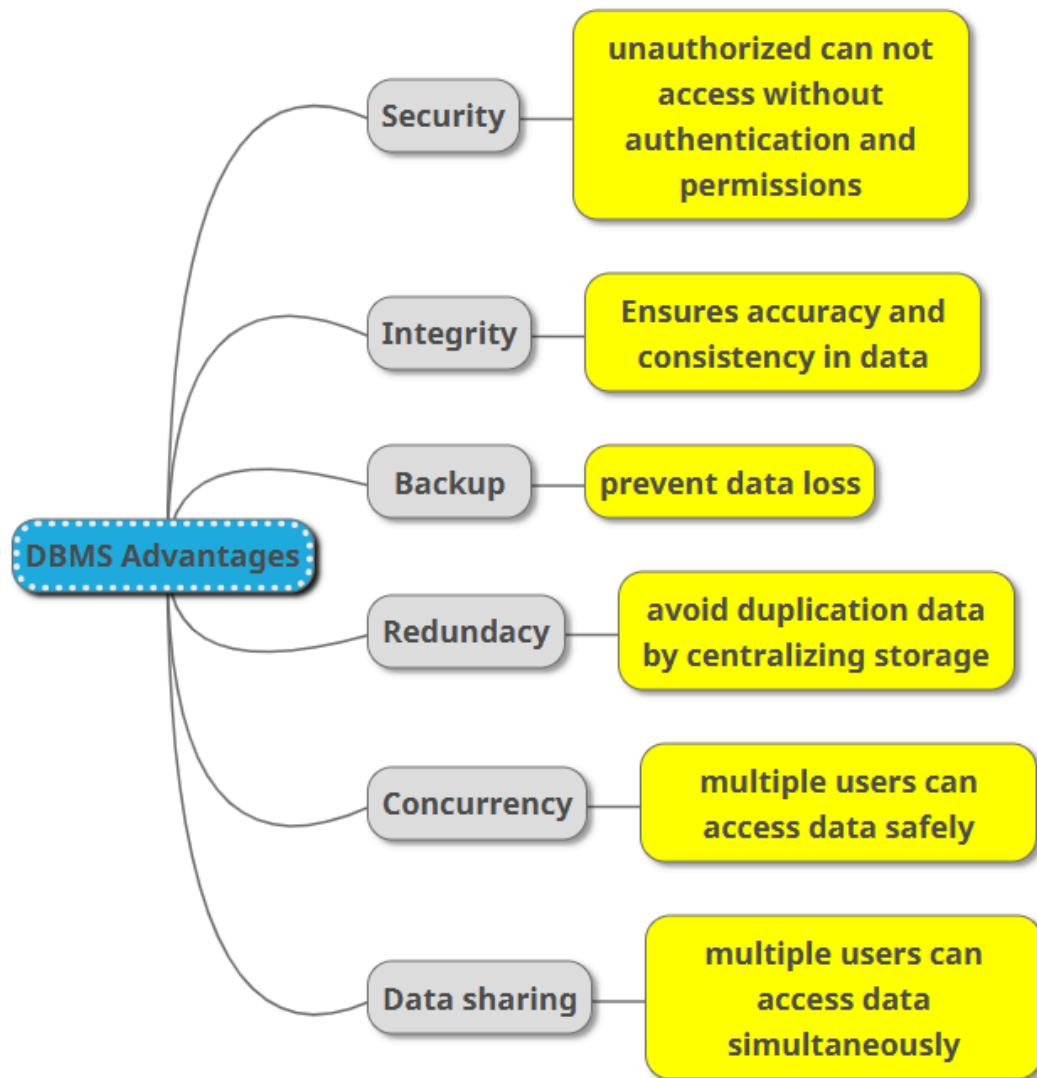


Database Search and Reporting Task

- **Comparison Between Flat File systems and Relational Databases:**

	Flat File systems	Relational Databases
Structure	Not structure	Structure
Data redundancy	Slow	Speed
Relationships	Not Relation	Relation
Examples	Store data to text files	Real database like Hospital database
Drawbacks	Not organize, not structure, not sharable	Complex, need management



- **Roles in a Database System:**

1. **System Analyst:** Can create specific systems for developers and designers by gathering user requirements and analyzing existing systems in general the connection between business needs and technical solutions.
2. **Database Designer:** Roles designs logical and physical database structure.
3. **Database Developer:** Implement database based on the design.
4. **Database Administrator:** manage and give the access permissions and role of the database.
5. **Application developer:** Depelops front-end or back-end that interact with databases.
6. **BI (Business Intelligence) Developer:** Transforms data into actionable insights for decision-making also builds reports using tools like Power BI.

- **Types of databases:**

Type	Relation(RDBMS)	Non-Relational(NoSQL)
Structure	Tables (rows & columns)	Collections, documents, key-value pairs, etc.
Schema	Fixed schema	Flexible
Examples	MySQL, PostgreSQL, Oracle	MongoDB (document), Cassandra (wide-column) Firebase
Best For	Structure data	Unstructure, fast development
Use case	Banking system	Real-time analytics

- **Centralized Database:**
 - Data stored in one location/server.
 - Easier to manage and secure.
 - Limitation: Single point of failure.
 - Use Case: Small businesses or local systems (e.g., school management systems).
- **Distributed Database:**
 - Data spread across multiple physical locations.
 - Locations may be connected via network.
 - Increases availability and reliability.
 - Use Case: Global financial systems, telecom networks.
- **Cloud Database:**
 - Hosted on cloud platforms (e.g., AWS, Azure, Google Cloud).
 - Scalable, accessible over the internet.
 - Managed services (e.g., Amazon RDS, Firebase).

→ Use Case: SaaS platforms, mobile apps, startups wanting quick deployment without infrastructure management.

- **Cloud Storage and Databases:**

☁ What is Cloud Storage?

- ★ is a service where data is stored on remote servers hosted on the internet (cloud) and managed by third-party providers.
- ★ Examples: Google Drive, Amazon S3, Microsoft OneDrive.
- ★ Stores files, images, videos, backups, and large datasets.

📊 How it relates to Databases:

- ❖ Cloud Storage = stores unstructured data like documents and media files.
- ❖ Cloud Database = stores structured or semi-structured data, optimized for querying and transactions.

Advantages and Disadvantages of Cloud-Based Databases:

✓ Advantages	✗ Disadvantages
Integration: Easy to connect with cloud-based tools, apps, and analytics.	Internet Dependency: Downtime or slow connection can interrupt service.
Cost-effective: Pay-as-you-go pricing; no need to maintain physical servers.	Unexpected costs if usage isn't monitored.
Accessibility: Access your database from anywhere with the internet.	Sensitive data stored offsite
Managed Services: Providers handle backups, updates, security, and uptime.	Less customization and control compared to on-premises DBs.
High Availability: Built-in failover and disaster recovery features.	Slight delay if data is hosted far from the user's location.

🔍 Use Case Examples:

- Azure SQL: Ideal for enterprise-grade applications in Microsoft ecosystems.
- Amazon RDS: Popular for scalable web apps and startups using AWS.
- Google Cloud Spanner: Best for global, distributed applications needing horizontal scaling.

- **Database Engines and Languages**

A Database Engine is the core service for storing, processing, and securing data.

Database Engine	Description	Language used
-----------------	-------------	---------------

MySQL	Open-source, widely used for web apps	ANSI SQL + MySQL Extensions
SQL Server	Microsoft's RDBMS	T-SQL (Transact-SQL)
Oracle	Enterprise-grade RDBMS	PL/SQL (Procedural Language SQL)
PostgreSQL	Advanced open-source RDBMS	ANSI SQL + PL/pgSQL

Is there a relationship between the engine and the language?

- SQL Server → T-SQL adds procedural programming, error handling, etc.
- Oracle → PL/SQL supports functions, triggers, and packages.
- PostgreSQL → PL/pgSQL allows custom functions and control-of-flow statements.

Can one language work across different engines?

- Basic ANSI SQL (e.g., simple SELECT, INSERT) works across all major engines.
- Advanced queries, stored procedures, and functions are often not portable:
- T-SQL won't work in Oracle without changes.
- PL/SQL won't run in MySQL or PostgreSQL.

Can We Transfer a Database Between Engines?

Is It Possible to Migrate Between Engines?

- SQL Server → MySQL using MySQL Workbench Migration Tool
- Oracle → PostgreSQL using Ora2Pg

Challenges of Engine-to-Engine Migration

Challenge	Description
Different SQL Dialects	T-SQL (SQL Server), PL/SQL (Oracle), etc., are not directly compatible with MySQL or PostgreSQL.
Data Type Mismatches	VARCHAR2 in Oracle → VARCHAR in PostgreSQL; DATETIME types may differ.

Stored Procedures & Triggers	Syntax and logic need to be rewritten in the target engine's language.
------------------------------	--

What to Consider Before Transferring

- **Schema Differences**

Tables, columns, and data types may need transformation.

- **Data Types Mapping**

Ensure each source data type has a valid equivalent in the target system.

- **Stored Procedures, Triggers, and Views**

Must be manually rewritten or converted using scripts/tools.

- **Business Logic**

Validate that logic implemented in stored code behaves the same after migration.

- **Encoding & Collation**

Character sets and sorting rules may affect data integrity.

- **Dependencies**

Applications, APIs, or reports depending on the database may break if not updated.

- **Testing & Validation**

Test migrated data and queries thoroughly before going live.

What is the Logical Schema?

A logical schema defines the structure of the data as understood by users and developers. It includes tables, columns, relationships, constraints — but without details about how they are physically stored.

What is the Physical Schema?

A physical schema describes how the data is actually stored in the database system. It includes file structures, indexing methods, partitions, data types specific to the DBMS, etc.

What's the Difference?

- Logical Schema → What data is stored and how it's related, Entities, attributes, relationships.
- Physical Schema → How data is stored internally, Storage, indexing, access paths, data files.

Why Is It Important to Understand Both?

- Logical design ensures correct and complete representation of the business requirements.
- Physical design ensures efficient performance and data integrity on the chosen platform.
- Together, they help build a system that is both functional and optimized.

Example: Student Entity

Logical Schema:

Entity: Student

- StudentID (Primary Key)
- FirstName
- LastName
- DateOfBirth
- Email
- MajorID (Foreign Key to Major table)

Physical Schema (for MySQL):

```
CREATE TABLE Student (
    StudentID INT PRIMARY KEY AUTO_INCREMENT,
    FirstName VARCHAR(50),
    LastName VARCHAR(50),
    DateOfBirth DATE,
    Email VARCHAR(100) UNIQUE,
    MajorID INT,
    INDEX idx_major (MajorID),
    FOREIGN KEY (MajorID) REFERENCES Major(MajorID)
);
```